5. HotStuff 암호화 시스템

5.1 암호화 시스템 개요

5.1.1 암호화의 역할

HotStuff에서 암호화는 다음과 같은 핵심 기능을 제공합니다:

- **인증**: 메시지가 올바른 송신자로부터 왔는지 확인
- **무결성**: 메시지가 전송 중 변조되지 않았는지 확인
- **집계**: 여러 서명을 하나로 결합하여 효율성 향상
- **검증**: 쿼럼 크기와 참여자 확인

5.1.2 지원하는 암호화 방식

```
┌─────────────────┐  ┌─────────────────┐
│       ECDSA     │  │      BLS12      │
│   (개별 서명)   │  │   (집계 서명)   │
│                 │  │                 │
│ • 단순 구현     │  │ • 효율적 집계   │
│ • 빠른 서명     │  │ • 작은 서명     │
│ • 큰 인증서     │  │ • 복잡한 구현   │
└─────────────────┘  └─────────────────┘
```

5.2 암호화 인터페이스 설계

5.2.1 고수준 Crypto 인터페이스

```
type Crypto interface {
// 부분 인증서 생성 및 검증
CreatePartialCert(block *hotstuff.Block) (hotstuff.PartialCert, error)
VerifyPartialCert(cert hotstuff.PartialCert) bool
// 쿼럼 인증서 생성 및 검증
CreateQuorumCert(block *hotstuff.Block, signatures []hotstuff.PartialCert) (hots
VerifyQuorumCert(qc hotstuff.QuorumCert) bool
// 타임아웃 인증서 생성 및 검증
CreateTimeoutCert(view hotstuff.View, timeouts []hotstuff.TimeoutMsg) (hotstuff.
VerifyTimeoutCert(tc hotstuff.TimeoutCert) bool
// AggregateQC 생성 및 검증 (Fast-HotStuff)
CreateAggregateQC(view hotstuff.View, timeouts []hotstuff.TimeoutMsg) (hotstuff.
VerifyAggregateQC(aggQC hotstuff.AggregateQC) (hotstuff.QuorumCert, bool)
}
```

5.2.2 저수준 CryptoBase 인터페이스

```
type CryptoBase interface {
// 기본 서명 기능
```

```
Sign(data []byte) (hotstuff.QuorumSignature, error)

Verify(signature hotstuff.QuorumSignature, data []byte) bool

// 서명 집계 기능

Combine(signatures ...hotstuff.QuorumSignature) (hotstuff.QuorumSignature, error

// 배치 검증 (선택적)

BatchVerify(signature hotstuff.QuorumSignature, data  map[hotstuff.ID][]byte) bo

}
```

5.3 Crypto 구현체

5.3.1 기본 구조

```
type crypto struct {

blockChain modules.BlockChain

configuration modules.Configuration

logger logging.Logger

modules.CryptoBase // 실제 암호화 구현

}


func  New(impl modules.CryptoBase) modules.Crypto {

return &crypto{CryptoBase: impl}

}
```

5.3.2 부분 인증서 생성

```
func (c crypto) CreatePartialCert(block *hotstuff.Block) (cert hotstuff.PartialC
// 블록 데이터에 서명
sig, err := c.Sign(block.ToBytes())
if err != nil {
return hotstuff.PartialCert{}, err
}
// 부분 인증서 생성
return hotstuff.NewPartialCert(sig, block.Hash()), nil
}
```

5.3.3 쿼럼 인증서 생성

```
func (c crypto) CreateQuorumCert(block *hotstuff.Block, signatures []hotstuff.Pa
// 제네시스 블록은 특별 처리
if block.Hash() == hotstuff.GetGenesis().Hash() {
return hotstuff.NewQuorumCert(nil, 0, hotstuff.GetGenesis().Hash()), nil
}
// 서명들을 추출
sigs := make([]hotstuff.QuorumSignature, 0, len(signatures))
for _, sig := range signatures {
sigs = append(sigs, sig.Signature())
```

```go
}

// 서명 집계

combinedSig, err := c.Combine(sigs...)

if  err != nil {

return hotstuff.QuorumCert{}, err

}

return  hotstuff.NewQuorumCert(combinedSig, block.View(), block.Hash()), nil

}
```

5.3.4 쿼럼 인증서 검증

```go
func (c crypto) VerifyQuorumCert(qc hotstuff.QuorumCert) bool {

// 제네시스 QC는 항상 유효

if  qc.BlockHash() == hotstuff.GetGenesis().Hash() {

return true

}

qcSignature := qc.Signature()

if  qcSignature == nil {

c.logger.DPanicf("quorum certificate has nil signature (view=%d)", qc.View())

return false

}

// 쿼럼 크기 확인
```

```
participants := qcSignature.Participants()
if  participants.Len() < c.configuration.QuorumSize() {
return false
}
// 블록 가져오기
block, ok := c.blockChain.Get(qc.BlockHash())
if !ok {
return false
}
// 서명 검증
return  c.Verify(qc.Signature(), block.ToBytes())
}
```

5.4 타임아웃 인증서 처리

5.4.1 타임아웃 인증서 생성

```
func (c crypto) CreateTimeoutCert(view hotstuff.View, timeouts []hotstuff.Timeou
// 뷰 0은 특별 처리
if  view == 0 {
return  hotstuff.NewTimeoutCert(nil, 0), nil
}
// 뷰 서명들 추출
```

```go
sigs := make([]hotstuff.QuorumSignature, 0, len(timeouts))
for _, timeout := range timeouts {
sigs = append(sigs, timeout.ViewSignature)
}
// 서명 집계
sig, err := c.Combine(sigs...)
if err != nil {
return hotstuff.TimeoutCert{}, err
}
return hotstuff.NewTimeoutCert(sig, view), nil
}
```

5.4.2 타임아웃 인증서 검증

```go
func (c crypto) VerifyTimeoutCert(tc hotstuff.TimeoutCert) bool {
// 뷰 0 TC는 항상 유효
if tc.View() == 0 {
return true
}
// 쿼럼 크기 확인
if tc.Signature().Participants().Len() < c.configuration.QuorumSize() {
return false
}
```

```
// 뷰 데이터에 대한 서명 검증
return c.Verify(tc.Signature(), tc.View().ToBytes())
}
```

5.5 Fast-HotStuff AggregateQC

5.5.1 AggregateQC 생성

```
func (c crypto) CreateAggregateQC(view hotstuff.View, timeouts []hotstuff.Timeou
qcs := make(map[hotstuff.ID]hotstuff.QuorumCert)
sigs := make([]hotstuff.QuorumSignature, 0, len(timeouts))
// 각 타임아웃 메시지에서 QC와 메시지 서명 추출
for _, timeout := range timeouts {
if qc, ok := timeout.SyncInfo.QC(); ok {
qcs[timeout.ID] = qc
}
if timeout.MsgSignature != nil {
sigs = append(sigs, timeout.MsgSignature)
}
}
// 메시지 서명들 집계
sig, err := c.Combine(sigs...)
if err != nil {
```

```
return hotstuff.AggregateQC{}, err
}
return  hotstuff.NewAggregateQC(qcs, sig, view), nil
}
```

5.5.2 AggregateQC 검증

```
func (c crypto) VerifyAggregateQC(aggQC hotstuff.AggregateQC) (highQC hotstuff.Q
messages := make(map[hotstuff.ID][]byte)
// 각 QC에서 최고 QC 찾기 및 메시지 재구성
for  id, qc := range  aggQC.QCs() {
if  highQC.View() < qc.View() || highQC == (hotstuff.QuorumCert{}) {
highQC = qc
}
// 타임아웃 메시지 재구성
messages[id] = hotstuff.TimeoutMsg{
ID: id,
View: aggQC.View(),
SyncInfo: hotstuff.NewSyncInfo().WithQC(qc),
}.ToBytes()
}
// 쿼럼 크기 확인
```

```
if  aggQC.Sig().Participants().Len() < c.configuration.QuorumSize() {
return hotstuff.QuorumCert{}, false
}
// 배치 서명 검증과 최고 QC 검증
if  c.BatchVerify(aggQC.Sig(), messages) && c.VerifyQuorumCert(highQC) {
return  highQC, true
}
return hotstuff.QuorumCert{}, false
}
```

5.6 ECDSA 구현

5.6.1 ECDSA 구조체

```
type ecdsaCrypto struct {
privKey *ecdsa.PrivateKey
pubKeys  map[hotstuff.ID]*ecdsa.PublicKey
ownID hotstuff.ID
}

type ecdsaSignature struct {
signer hotstuff.ID
sig []byte
```

```go
}


func (es ecdsaSignature) Participants() hotstuff.IDSet {
set := hotstuff.NewIDSet()
set.Add(es.signer)
return set
}
```

5.6.2 ECDSA 서명 생성

```go
func (ec *ecdsaCrypto) Sign(data []byte) (hotstuff.QuorumSignature, error) {
// 데이터 해시
hash := sha256.Sum256(data)
// ECDSA 서명 생성
r, s, err := ecdsa.Sign(rand.Reader, ec.privKey, hash[:])
if err != nil {
return nil, err
}
// 서명 직렬화
sig := append(r.Bytes(), s.Bytes()...)
return ecdsaSignature{
signer: ec.ownID,
sig: sig,
```

```
}, nil

}
```

5.6.3 ECDSA 서명 검증

```
func (ec *ecdsaCrypto) Verify(signature hotstuff.QuorumSignature, data []byte) b

ecdsaSig, ok := signature.(ecdsaSignature)

if !ok {

return false

}

// 공개키 가져오기

pubKey, exists := ec.pubKeys[ecdsaSig.signer]

if !exists {

return false

}

// 서명 파싱

if  len(ecdsaSig.sig) != 64 {

return false

}

r := new(big.Int).SetBytes(ecdsaSig.sig[:32])

s := new(big.Int).SetBytes(ecdsaSig.sig[32:])

// 데이터 해시

hash := sha256.Sum256(data)
```

```
// 서명 검증
return ecdsa.Verify(pubKey, hash[:], r, s)
}
```

5.6.4 ECDSA 서명 집계 (배열 방식)

```
type ecdsaMultiSignature struct {
signatures []ecdsaSignature
}

func (ec *ecdsaCrypto) Combine(signatures ...hotstuff.QuorumSignature) (hotstuff
var ecdsaSigs []ecdsaSignature
for _, sig := range signatures {
if ecdsaSig, ok := sig.(ecdsaSignature); ok {
ecdsaSigs = append(ecdsaSigs, ecdsaSig)
} else if multiSig, ok := sig.(ecdsaMultiSignature); ok {
ecdsaSigs = append(ecdsaSigs, multiSig.signatures...)
} else {
return nil, fmt.Errorf("invalid signature type")
}
}
return ecdsaMultiSignature{signatures: ecdsaSigs}, nil
}
```

```go
func (ems ecdsaMultiSignature) Participants() hotstuff.IDSet {
set := hotstuff.NewIDSet()
for _, sig := range ems.signatures {
set.Add(sig.signer)
}
return set
}
```

5.7 BLS12-381 구현

5.7.1 BLS 구조체

```go
type blsCrypto struct {
privKey bls.SecretKey
pubKeys map[hotstuff.ID]bls.PublicKey
ownID hotstuff.ID
threshold int
}

type blsSignature struct {
sig bls.Signature
participants hotstuff.IDSet
```

```
}
```

5.7.2 BLS 서명 생성

```
func (bc *blsCrypto) Sign(data []byte) (hotstuff.QuorumSignature, error) {
// BLS 서명 생성
sig := bc.privKey.Sign(data)
// 참여자 집합 생성
participants := hotstuff.NewIDSet()
participants.Add(bc.ownID)
return blsSignature{
sig: sig,
participants: participants,
}, nil
}
```

5.7.3 BLS 서명 집계

```
func (bc *blsCrypto) Combine(signatures ...hotstuff.QuorumSignature) (hotstuff.Q
var  blsSigs []bls.Signature
participants := hotstuff.NewIDSet()
for  _, sig := range  signatures {
blsSig, ok := sig.(blsSignature)
```

```go
if !ok {
return  nil, fmt.Errorf("invalid signature type")
}
blsSigs = append(blsSigs, blsSig.sig)
blsSig.participants.ForEach(func(id hotstuff.ID) {
participants.Add(id)
})
}
// BLS 서명 집계
aggregated := bls.AggregateSignatures(blsSigs)
return blsSignature{
sig: aggregated,
participants: participants,
}, nil
}
```

5.7.4 BLS 서명 검증

```go
func (bc *blsCrypto) Verify(signature hotstuff.QuorumSignature, data []byte) boo
blsSig, ok := signature.(blsSignature)
if !ok {
return false
}
```

```go
// 참여자들의 공개키 수집
var pubKeys []bls.PublicKey
blsSig.participants.ForEach(func(id hotstuff.ID) {
if pubKey, exists := bc.pubKeys[id]; exists {
pubKeys = append(pubKeys, pubKey)
}
})
// 공개키 집계
aggregatedPubKey := bls.AggregatePublicKeys(pubKeys)
// 서명 검증
return blsSig.sig.Verify(data, aggregatedPubKey)
}
```

5.8 배치 검증 최적화

5.8.1 배치 검증 인터페이스

```go
func (bc *blsCrypto) BatchVerify(signature hotstuff.QuorumSignature, data map[h
blsSig, ok := signature.(blsSignature)
if !ok {
return false
}
// 각 참여자별 데이터와 공개키 수집
```

```go
var messages [][]byte
var pubKeys []bls.PublicKey
blsSig.participants.ForEach(func(id hotstuff.ID) {
if msg, exists := data[id]; exists {
if pubKey, exists := bc.pubKeys[id]; exists {
messages = append(messages, msg)
pubKeys = append(pubKeys, pubKey)
}
}
})
// 배치 검증 수행
return bls.BatchVerify(blsSig.sig, messages, pubKeys)
}
```

5.9 암호화 성능 최적화

5.9.1 서명 캐싱

```go
type cachedCrypto struct {
modules.CryptoBase
signatureCache map[string]hotstuff.QuorumSignature
verifyCache map[string]bool
mutex sync.RWMutex
```

```
}

func (cc *cachedCrypto) Sign(data []byte) (hotstuff.QuorumSignature, error) {
key := string(data)
cc.mutex.RLock()
if sig, exists := cc.signatureCache[key]; exists {
cc.mutex.RUnlock()
return sig, nil
}
cc.mutex.RUnlock()
sig, err := cc.CryptoBase.Sign(data)
if err != nil {
return nil, err
}
cc.mutex.Lock()
cc.signatureCache[key] = sig
cc.mutex.Unlock()
return sig, nil
}
```

5.9.2 병렬 검증

```
func (bc *blsCrypto) ParallelVerify(signatures []hotstuff.QuorumSignature, data
```

```go
results := make([]bool, len(signatures))
var wg sync.WaitGroup
for i, sig := range signatures {
wg.Add(1)
go func(idx int, signature hotstuff.QuorumSignature, message []byte) {
defer wg.Done()
results[idx] = bc.Verify(signature, message)
}(i, sig, data[i])
}
wg.Wait()
return results
}
```

5.10 보안 고려사항

5.10.1 키 관리

- **키 생성**: 안전한 난수 생성기 사용
- **키 저장**: 하드웨어 보안 모듈(HSM) 또는 안전한 키 저장소
- **키 교체**: 주기적인 키 로테이션 지원

5.10.2 사이드 채널 공격 방어

- **타이밍 공격**: 상수 시간 알고리즘 사용

- **전력 분석**: 하드웨어 수준 보호
- **캐시 공격**: 캐시 독립적 구현

5.10.3 양자 내성

- **포스트 양자 암호**: 미래 양자 컴퓨터 대비
- **하이브리드 방식**: 기존 암호와 양자 내성 암호 병용
- **업그레이드 경로**: 암호화 방식 변경 지원

이러한 암호화 시스템을 통해 HotStuff는 비잔틴 환경에서도 안전하고 효율적인 합의를 달성할 수 있습니다.